EXAMEN PARCIAL 02

Tema: PARTE II

Nota

Estudiante	Escuela	Asignatura
Camilo B.A. Capatinta Almanza (Teoría grupo B) ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Arq. de computadoras Semestre: IV Código: 1702227

Laboratorio	Tema	Duración
02	PARTE II	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	08/11/2026	08/11/2026

Índice

1. **Pregunta 1: Decimal a IEEE 754 de precisión simple** **3**
2. **Pregunta 2: IEEE 754 de precisión simple a decimal** **3**
3. **Pregunta 3: Memoria jerárquica** **3**
4. **Pregunta 4: Ejecución de la instrucción `SW #2AB`** **5**
5. **Pregunta 5: Franjas horizontales de colores en assembler Intel** **6**
6. **Pregunta 6: Manipulación de bits del registro CX** **7**

1. Pregunta 1: Decimal a IEEE 754 de precisión simple

Se representa $13{,}15625_{10}$ en el formato IEEE 754 de precisión simple (32 bits: 1 bit de signo, 8 bits de exponente con sesgo 127 y 23 bits de mantisa).

- **Signo:** el número es positivo, por lo tanto $S = 0$.
- **Parte entera:** $13_{10} = 1101_2$.
- **Parte fraccionaria:** se multiplica por 2 de forma sucesiva y se toma la parte entera de cada resultado.

Operación	Resultado	Bit
$0{,}15625 \times 2$	0,3125	0
$0{,}3125 \times 2$	0,625	0
$0{,}625 \times 2$	1,25	1
$0{,}25 \times 2$	0,5	0
$0{,}5 \times 2$	1,0	1

- Entonces $0{,}15625_{10} = 0{,}00101_2$ y $13{,}15625_{10} = 1101{,}00101_2$.
- **Normalización:** $1101{,}00101_2 = 1{,}10100101_2 \times 2^3$ (la coma se desplaza 3 lugares a la izquierda), es decir, $e = 3$.
- **Exponente sesgado:** $E = e + 127 = 3 + 127 = 130_{10} = 10000010_2$.
- **Mantisa:** son los bits que siguen al 1 implícito, 10100101_2, completados con ceros hasta los 23 bits.

Signo (1 bit)	Exponente (8 bits)	Mantisa (23 bits)
0	10000010	10100101000000000000000

Respuesta: $13{,}15625_{10} = 0\ 10000010\ 10100101000000000000000_2 = 41528000_{16}$.
Comprobación: $1{,}10100101_2 = 1 + \frac{1}{2} + \frac{1}{8} + \frac{1}{64} + \frac{1}{256} = 1{,}64453125$ y $1{,}64453125 \times 2^3 = 13{,}15625$.

2. Pregunta 2: IEEE 754 de precisión simple a decimal

Número dado: 1 01100110 10100000000000000000000 (signo, exponente y mantisa).

- **Signo:** $S = 1$, por lo tanto el número es negativo.
- **Exponente:** $E = 01100110_2 = 64 + 32 + 4 + 2 = 102_{10}$, entonces $e = E - 127 = 102 - 127 = -25$.
- **Mantisa:** $M = 101\,000\ldots0_2$, por lo que el significando (con el 1 implícito) es $1{,}101_2 = 1 + \frac{1}{2} + \frac{1}{8} = 1{,}625$.

$$N = (-1)^S \times 1.M \times 2^e = -1{,}625 \times 2^{-25} = -\frac{13}{2^{28}} = -\frac{13}{268\,435\,456}$$

Respuesta: $N \approx -4{,}8429 \times 10^{-8}$ (en hexadecimal, $B3500000_{16}$).

3. Pregunta 3: Memoria jerárquica

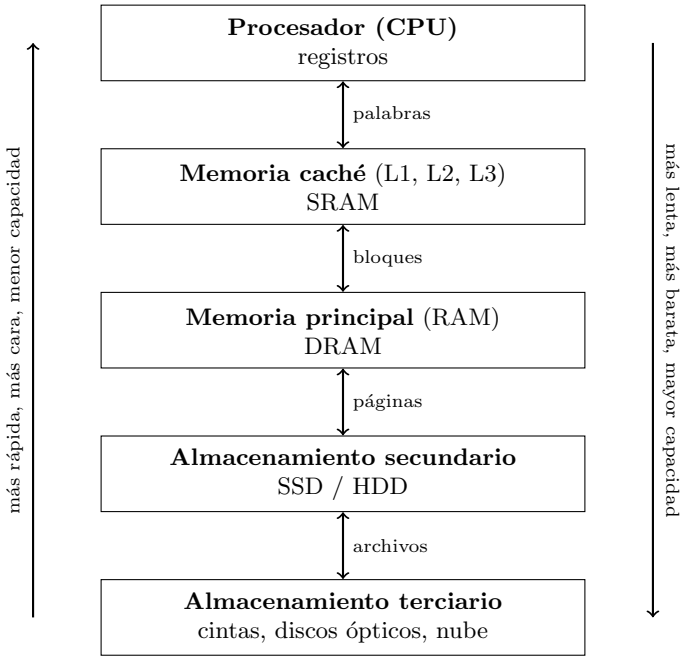
La **memoria jerárquica** organiza la memoria del computador en varios niveles construidos con tecnologías distintas. Los niveles cercanos al procesador son pequeños, muy rápidos y caros por bit; los niveles lejanos son grandes, lentos y baratos. Como ninguna tecnología es a la vez rápida, barata y de gran capacidad, se combinan para que el sistema se comporte como una memoria grande y casi tan rápida como la más veloz.

La jerarquía funciona gracias al **principio de localidad**:

- **Localidad temporal:** un dato usado recientemente tiende a volver a usarse pronto.

- **Localidad espacial:** después de acceder a una dirección es probable acceder a las cercanas, por eso se copian bloques completos y no palabras sueltas.

Cuando el procesador necesita un dato, lo busca primero en el nivel más cercano. Si está (acierto) lo usa; si no está (fallo), se trae un bloque desde el nivel siguiente. Cada nivel contiene un subconjunto de los datos del nivel inferior.



Esquema de un procesador conectado a una memoria jerárquica.

Niveles que se encuentran:

- **Registros de la CPU:** están dentro del procesador y guardan operandos y resultados inmediatos. Son el almacenamiento más rápido, pero muy pocos.
- **Memoria caché (L1, L2, L3):** memoria SRAM entre la CPU y la RAM que guarda copias de los bloques más usados. La administra el hardware; L1 es la más pequeña y rápida, y L3 la más grande y lenta.
- **Memoria principal (RAM):** memoria DRAM que contiene los programas y datos en ejecución. El sistema operativo la administra con memoria virtual (paginación y MMU).
- **Secundario (SSD o HDD):** memoria no volátil que guarda archivos y programas y recibe las páginas que no caben en la RAM.
- **Almacenamiento terciario:** cintas, discos ópticos o la nube, para copias de seguridad y datos que casi no se usan.

Nivel	Tecnología	Acceso	Capacidad	Gestión
Registros	Flip-flops	< 1 ns	cientos de bytes	Compilador
Caché (L1–L3)	SRAM	1 a 30 ns	KB a MB	Hardware
Memoria principal	DRAM	50 a 100 ns	GB	S.O.
Secundario	SSD / HDD	0.1 a 10 ms	GB a TB	S.O.
Terciario	Cinta / nube	s a min	TB a PB	Operador

Valores típicos y orientativos (S.O. = sistema operativo).

Al bajar en la jerarquía aumentan la capacidad y el tiempo de acceso, y disminuye el costo por bit.

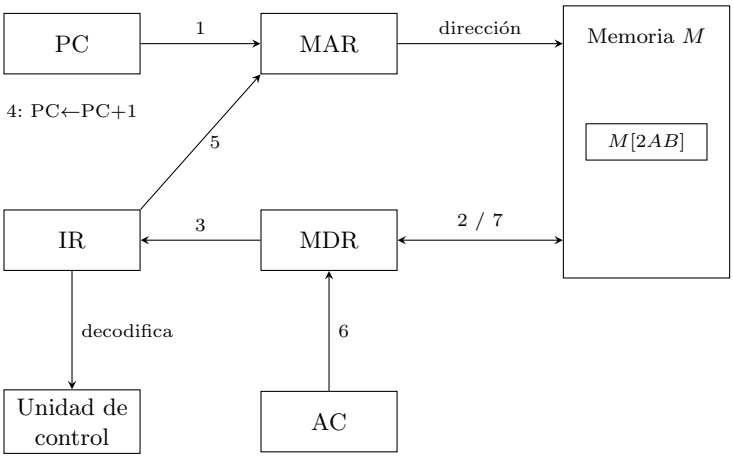
4. Pregunta 4: Ejecución de la instrucción `SW #2AB`

La instrucción `SW #2AB` (*store word*) guarda el contenido del acumulador en la posición de memoria $2AB_{16}$ (el símbolo # indica que $2AB$ es una dirección de memoria). En notación de transferencia de registros:

$$M[2AB] \leftarrow AC$$

Como toda instrucción del computador elemental, se ejecuta en tres fases: búsqueda (*fetch*), decodificación y ejecución. Los elementos que intervienen son:

- **PC (contador de programa):** contiene la dirección de la siguiente instrucción.
- **MAR (registro de dirección de memoria):** coloca la dirección en el bus de direcciones.
- **MDR (registro de datos de memoria):** recibe o entrega el dato que viaja por el bus de datos.
- **IR (registro de instrucción):** guarda la instrucción que se está ejecutando.
- **AC (acumulador):** contiene el dato que se va a almacenar.
- **Unidad de control (UC):** decodifica la instrucción y genera las señales de control.



Registros y transferencias que intervienen (los números indican el orden de las micro-operaciones).

Paso	Micro-operación	Qué ocurre
Búsqueda (*fetch*)		
1	$MAR \leftarrow PC$	La dirección de la instrucción se coloca en MAR.
2	$MDR \leftarrow M[MAR]$	Lectura de memoria: la instrucción llega a MDR.
3	$IR \leftarrow MDR$	La instrucción se guarda en el IR.
4	$PC \leftarrow PC + 1$	El PC apunta a la siguiente instrucción.
Decodificación		
–	Decodificar IR	La UC reconoce el código `SW` y separa el campo de dirección ($2AB$).
Ejecución		
5	$MAR \leftarrow IR$[dir]	La dirección de destino ($2AB$) se coloca en MAR.
6	$MDR \leftarrow AC$	El dato por guardar se coloca en MDR.
7	$M[MAR] \leftarrow MDR$	Escritura: queda $M[2AB] = AC$.

Los pasos 1 a 4 son comunes a todas las instrucciones; el paso 4 puede hacerse al mismo tiempo que la lectura de memoria. Al terminar, la posición $2AB$ contiene una copia del acumulador (por ejemplo, si $AC = 5$ entonces $M[2AB] = 5$), el acumulador y las banderas no cambian, y el PC ya apunta a la siguiente instrucción.

POR HACER (borrar este cuadro al terminar): Compara los nombres de los registros y el formato de instrucción con los apuntes del computador elemental de tu curso. Aquí se usaron MAR y MDR (en algunos apuntes se llaman RDM y RDD, o AR y DR) y se supuso una palabra de instrucción con código de operación y un campo de dirección de 12 bits ($2AB_{16} = 0010\ 1010\ 1011_2$). Si difieren, cambia solo los nombres o el tamaño del campo; la secuencia de micro-operaciones es la misma. Si en clase usaron señales de control, agrégalas en cada paso.

5. Pregunta 5: Franjas horizontales de colores en assembler Intel

Idea. La pantalla de texto de 80×25 caracteres dispone de 16 colores (numerados de 0 a 15). La imagen del enunciado muestra 16 franjas, una por color: cada franja ocupa una fila y 16 columnas, y la primera es negra (por eso no se distingue del fondo). Se usan dos funciones de la interrupción `INT 10h` del BIOS:

- `AH = 02h`: posiciona el cursor (`DH` = fila, `DL` = columna, `BH` = página).
- `AH = 09h`: escribe el carácter de `AL` con el atributo de `BL`, repetido `CX` veces desde la posición del cursor.

Cada franja se dibuja con el carácter de bloque lleno (`0DBh`) cuyo color de primer plano es el número de la fila. De este modo, la fila n se pinta con el color n y no hace falta una tabla de colores.

Fila	Color	Nombre	Fila	Color	Nombre
0	0h	Negro	8	8h	Gris oscuro
1	1h	Azul	9	9h	Azul claro
2	2h	Verde	10	Ah	Verde claro
3	3h	Cian	11	Bh	Cian claro
4	4h	Rojo	12	Ch	Rojo claro
5	5h	Magenta	13	Dh	Magenta claro
6	6h	Marrón	14	Eh	Amarillo
7	7h	Gris claro	15	Fh	Blanco

Programa (`franjas.asm`):

```
; franjas.asm - 16 franjas horizontales de colores (texto 80x25)
; Ensamblar: nasm -f bin franjas.asm -o franjas.com
org 100h

    mov ax, 0003h       ; modo texto 80x25 (limpia la pantalla)
    int 10h

    mov dh, 0           ; DH = fila actual = numero de color (0..15)
siguiente:
    mov ah, 02h         ; funcion 02h: posicionar el cursor
    mov bh, 0           ; pagina de video 0
    mov dl, 0           ; columna 0 (la fila ya esta en DH)
    int 10h

    mov ah, 09h         ; funcion 09h: escribir caracter con atributo
    mov al, 0DBh        ; caracter de bloque lleno
    mov bl, dh          ; atributo = color de la franja
    mov cx, 16          ; 16 repeticiones = ancho de la franja
    int 10h

    inc dh              ; siguiente fila y siguiente color
    cmp dh, 16          ; ya se dibujaron las 16 franjas?
    jne siguiente

    mov ah, 00h         ; esperar una tecla para ver el resultado
    int 16h

    mov ax, 4C00h       ; terminar el programa
    int 21h
```

Funcionamiento.

- Se fija el modo de texto 80×25 con `AX = 0003h`, lo que también limpia la pantalla.

- `DH` lleva la fila y, a la vez, el número de color (0 a 15). En cada vuelta se posiciona el cursor en la columna 0 de esa fila.
- Se escriben 16 bloques llenos con el atributo `BL = DH`, es decir, una franja de 16 columnas del color de esa fila.
- Se incrementa `DH` y se repite hasta completar 16 franjas (filas 0 a 15).
- Se espera una tecla (`INT 16h`) para poder ver la pantalla y se termina con `INT 21h, AH = 4Ch`.

Se usa el color de primer plano porque funciona igual en cualquier entorno; los colores de fondo 8 a 15 dependen de si el modo de parpadeo del texto está activado.

POR HACER (borrar este cuadro al terminar): Ensambla (`nasm -f bin franjas.asm -o franjas.com`), ejecuta `franjas.com` en DOSBox y toma una captura de pantalla para adjuntarla aquí. Comprueba que se vean las 16 filas en el mismo orden que la imagen del enunciado. El programa se probó en un emulador de 16 bits y dibuja las 16 filas con los colores 0 a 15, pero no se pudo ver en DOSBox ni en EMU8086 desde aquí.

6. Pregunta 6: Manipulación de bits del registro CX

Se pide, sobre el dato de CX (bits D7 a D0): invertir los bits impares D5, D3 y D1; borrar los bits pares D6, D4 y D2; y poner en 1 los bits D7 y D0. Cada acción se resuelve con una instrucción lógica y una máscara:

- **Invertir** bits: `XOR` con 1 en los bits a invertir. Máscara $0010\ 1010_2 = 2Ah$ (D5, D3, D1).
- **Borrar** bits: `AND` con 0 en los bits a borrar y 1 en el resto. Máscara $1010\ 1011_2 = ABh$ (D6, D4, D2 en cero). Como CX tiene 16 bits, se usa $FFABh$ para no alterar el byte alto.
- **Poner en 1** bits: `OR` con 1 en los bits a poner. Máscara $1000\ 0001_2 = 81h$ (D7 y D0).

Bit	D7	D6	D5	D4	D3	D2	D1	D0
Acción	poner 1	borrar	invertir	borrar	invertir	borrar	invertir	poner 1

Las tres operaciones afectan a bits distintos, por lo que el orden en que se aplican no cambia el resultado.

Programa (`bits.asm`), con el dato de ejemplo del enunciado ($CX = 00B1h$, es decir, $1011\ 0001_2$ en D7 a D0):

```
; bits.asm - manipulacion de bits del registro CX
; Ensamblar: nasm -f bin bits.asm -o bits.com
org 100h

    mov cx, 00B1h       ; dato de ejemplo: D7..D0 = 1011 0001
    xor cx, 002Ah       ; invierte D5, D3 y D1      (0010 1010)
    and cx, 0FFABh      ; borra D6, D4 y D2         (1010 1011)
    or  cx, 0081h       ; pone en 1 D7 y D0         (1000 0001)

    ; --- mostrar el byte bajo de CX en binario (opcional) ---
    mov bx, 8           ; 8 bits por imprimir
imprimir:
    rol cl, 1           ; el bit D7 pasa al acarreo (CF)
    mov dl, '0'
    adc dl, 0           ; DL = '0' + CF  ->  '0' o '1'
    mov ah, 02h         ; imprimir el caracter que esta en DL
    int 21h
    dec bx
    jnz imprimir

    mov ax, 4C00h       ; terminar el programa
    int 21h
```

Las cuatro primeras instrucciones son la solución; el bloque `imprimir` solo sirve para ver el resultado en DOSBox, donde no hay ventana de registros.

Comprobación con el ejemplo:

Paso	**Efecto**	**Byte bajo de CX**
Dato inicial	—	$1011\ 0001_2 = B1h$
`XOR CX, 002Ah`	invierte D5, D3 y D1	$1001\ 1011_2 = 9Bh$
`AND CX, 0FFABh`	borra D6, D4 y D2	$1000\ 1011_2 = 8Bh$
`OR CX, 0081h`	pone en 1 D7 y D0	$1000\ 1011_2 = 8Bh$

En este ejemplo el `OR` no cambia el valor porque D7 y D0 ya estaban en 1.

Respuesta: el resultado es $CX = 008Bh$, es decir, $1000\ 1011_2$ en D7 a D0. Para cualquier otro dato el byte alto no cambia; por ejemplo, $CX = 1234h$ queda en $128Bh$.

POR HACER (borrar este cuadro al terminar): Ensambla (`nasm -f bin bits.asm -o bits.com`), ejecuta `bits.com` en DOSBox y verifica que imprime `10001011`; agrega la captura. El programa se probó en un emulador de 16 bits y da ese resultado, pero no se pudo ejecutar en DOSBox desde aquí.